CAPÍTULO 3

ESTRUCTURA DE UN PROGRAMA EN ENSAMBLADOR

Programas En Ensamblador

El **lenguaje ensamblador** es un lenguaje que utiliza símbolos en lugar de instrucciones codificadas en lenguaje máquina. También permite hacer referencia a las localidades de memoria por nombres simbólicos en lugar de usar sus direcciones directamente. Un programa escrito en lenguaje ensamblador, llamado código fuente, es traducido al código máquina de un microprocesador mediante un programa llamado **ensamblador**.

Un programa en ensamblador consiste en una secuencia de caracteres almacenada en uno o más archivos que serán traducidos por el ensamblador al lenguaje máquina de una determinada computadora.

En este curso se estudiará el lenguaje ensamblador del procesador 8086 y se utilizará el ensamblador Turbo Assembler de la compañía Borland Int.

Palabras Reservadas

Las palabras reservadas son para uso estricto del ensamblador. Éstas incluyen a los operadores, directivas y símbolos predefinidos. Estas palabras no pueden usarse como identificadores. La tabla 3.1 muestra la lista de palabras reservadas de Turbo Assembler:

Tabla 3.1 Palabras reservadas de Turbo Assembler.

:	;	=	?	[]	/
()	+	-	*	.	.186
.286	.286c	.286p	.287	.386	.386c
.387	.8086	.8087	align	.alpha	and
arg	assume	%bin.	byte	catstr	@code
codeseg	@codesize	comm	comment	%conds	.const
const	@cpu	%cref	.cref	%crefall	%crefref

Tabla 3.1 Palabras reservadas de Turbo Assembler (Cont.)

%crefuref	%ctls	@curseg	@data	.data	.data?
dataptr	dataseg	@datasize	??date	db	dd
%depth	df	display	dosseg	dp	dq
dt	dup	dw	dword	else	elseif
Emul	End	Endif	Endm	Endp	Ends
Eq	Equ	Err	.err	.err1	.err2
.errb	.errdef	Errdif	errdifi	erre	erridn
erridni	errifnb	Errifndef	errnb	errndef	errnz
even	evendata	Exitm	extrn	far	fardata
@fardata	.fardata	@fardata?	.fardata?	@filename	??filename
fword	ge	Global	group	gt	high
ideal	if	If1	if2	ifb	ifdef
ifdif	ifdifi	Ife	ifidn	ifidni	ifnb
ifndef	%incl	Include	includelib	instr	irp
irpc	jumps	Label	.lall	large	le
length	.lfcond	%linum	%list	.list	local
locals	low	lt	macro	%macs	mask
masm	masm51	mod	model	.model	multerrs
name	ne	near	%newpage	%noconds	%nocref
%noctls	noemul	%noincl	nojumps	%nolist	nolocals
%nomacs	nomasm51	nomulterrs	%nosyms	not	nothing
%notrunc	nowarn	offset	or	org	%out
p186	p286	p286n	p287	p386	p386n
p386p	p387	p8086	p8087	page	%pagesize
%pcnt	pno87	%poplctl	proc	ptr	public
purge	%pushlctl	pwor	quirqs	qword	radix
.radix	record	rept	.sall	seg	segment
.seq	.sfcond	shl	short	shr	size
sizestr	small	stack	.stack	struc	substr
subttl	%subttl	%syms	symtype	%tabsize	tbyte
%text	.tfcond	this	??time	title	%title
%trunc	type	.type	udataseg	ufardata	union
unknown	uses	??version	warn	width	word
@wordsize	.xall	.xcref	.xlist	xor	

Conjunto De Instrucciones Del 8086

El conjunto de instrucciones del lenguaje ensamblador son mnemónicos de las instrucciones en lenguaje máquina del procesador. El conjunto completo de instrucciones de la familia de procesadores 8086 es:

Tabla 3.2 Conjunto de Instrucciones del 8086

aaa	aad	aam	aas	adc	add
cmc	cmp	cmps	cmpsb	cmpsw	cwd
daa	das	dec	div	esc	hlt
idiv	imul	in	inc	int	into
iret	ja	jae	jb	jbe	jc
jcxz	je	jg	jge	jl	jle
jmp	jna	jnae	jnb	jnbe	jnc
jne	jng	jnge	jnl	jnle	jno
jnp	jns	jnz	jo	jp	jpe
jpo	js	jz	lahf	lds	lea
les	lock	lods	lodsb	lodsw	loop
loope	loopne	loopnz	loopw	loopwe	loopwz
loopwne	loopwnz	loopz	mov	movs	movsb
movsw	mul	neg	nop	not	or
out	pop	popf	push	pushf	rcl
rcr	rep	repe	repne	repnz	repz
ret	retn	retf	rol	ror	sahf
sal	sar	sbb	scas	scasb	scasw
segcs	segds	segss	seges	shl	shr
stc	std	sti	stos	stosb	stosw
stosw	test	wait	xchg	xlat	xlatb
xor					

Estructura De Un Programa En Ensamblador

Un programa en el lenguaje ensamblador consiste de una secuencia de proposiciones, **una en cada línea** del código fuente.

La sintaxis de una proposición es la siguiente:

`[`*`etiqueta`*`] [`*`instrucción`*`|`*`directiva`* `[`*`operandos`*`]] [;`*`comentario`*`]`

donde *etiqueta, instrucción*|*directiva*, *operandos*, *comentario* son los campos de la proposición. Los campos se separan por caracteres blancos: caracteres de espacio y/o de tabulación. Todos los campos son opcionales, pero el campo *operandos* sólo pueden estar presente si existe el campo *instrucción*|*directiva.*

etiqueta es un nombre simbólico empleado para referirse a números, cadenas de caracteres o localidades de memoria dentro de un programa. Las etiquetas permiten darle nombre a las variables, constantes y localidades de una instrucción particular.

- Las etiquetas pueden contener los siguientes caracteres: A-Z a-z _ @ $? 0-9. Los dígitos 0-9 no pueden usarse como el primer carácter de una etiqueta. Un sólo carácter $ o ? no puede usarse como etiqueta ya que tienen un significado especial.

- Las etiquetas deben ser únicas. Con excepción de las definidas con la directiva =, y las etiquetas locales de macros y subrutinas. Las etiquetas pueden usarse como operandos tantas veces como se desee.

- Una etiqueta puede aparecer por sí sola en una línea. En este caso el valor asociado a la etiqueta es la dirección de la operación en la siguiente línea en el programa.

- Las palabras reservadas no pueden usarse como etiquetas.

- Las etiquetas que aparecen solas en una línea y las que van seguidas de una instrucción deben terminar en dos puntos (:). Las demás etiquetas por lo general no terminan en dos puntos.

instrucción es un mnemónico de una instrucción del procesador. Cada instrucción se traduce directamente a una instrucción del lenguaje máquina del microprocesador.

directiva es una instrucción para el ensamblador. No produce código ejecutable, sino que controla varios de los aspectos de cómo opera el ensamblador: el tipo de código generado (8086, 80286, 80386, etc.), los segmentos a usar, etc.

operandos consiste de cero, uno o más operandos. Un operando le especifica al ensamblador qué valor, registro, localidad de memoria, etc., asociar con cada instrucción.

- Hay varias clases de operandos: registros, constantes, etiquetas, variables y cadenas.

- El tipo y número de operandos depende de la instrucción o directiva.

- Si hay dos o más, van separados por comas (,)

comentario es cualquier secuencia de caracteres precedidos por un punto y coma (;) y que termina con el fin de la línea.

- Los comentarios son ignorados por el ensamblador

- Las líneas en blanco se tratan como comentarios.

Por ejemplo considere el programa de la siguiente página. Los números a la izquierda del recuadro no forman parte del programa y sólo sirven para numerar las líneas:

> Las líneas 1 a 7, 9, 16, 20, 28 y 45 son líneas con sólo comentarios.
>
> Las líneas 8, 10, 15, 17, 19, 21, 27, 29, 34, 39, 44 y 46 son líneas en blanco y son tratadas como comentarios. Se usan para separar las partes de un programa y darle mayor claridad.
>
> Las líneas 31 y 40 son líneas con sólo una etiqueta

Las líneas 32, 33, 35 a 38 y 41 a 43 contienen instrucciones.

Las líneas 11 a 14, 18, 22 a 26, 30 y 47 contienen directivas.

```
 1 ;*******************************************************
 2 ; PRIMER.ASM.
 3 ;
 4 ; Este programa ilustra la estructura de un programa en
 5 ; ensamblador. También muestra las principales directivas
 6 ; para El ensamblador.
 7 ;*******************************************************
 8
 9 ;****** CÓDIGO DE INICIO **********************************
10
11         ideal
12         dosseg
13         model small
14         stack 256
15
16 ;****** DECLARACIÓN DE CONSTANTES SIMBÓLICAS **************
17
18 cte      equ     10
19
20 ;****** VARIABLES DEL PROGRAMA ****************************
21
22         dataseg
23 codsal  db      0
24 dato1   db      ?
25 dato2   db      ?
26 resul   db      ?
27
28 ;****** CÓDIGO DEL PROGRAMA *******************************
29
30 codeseg
31 inicio:
32         mov     ax, @data    ; Inicializa el segmento de
33         mov     ds, ax       ; datos
34
35         mov     al, [dato1]  ; resul = dato1 + dato2 + cte
36         add     al, [dato2]
37         add     al, cte
38         mov     [resul], al
39
40 salir:
41         mov     ah, 04Ch
42         mov     al, [codsal]
43         int     21h
44
45 ;****** CÓDIGO DE TERMINACIÓN ******************************
46
47         end    inicio
```

Normalmente podemos considerar que un programa en ensamblador está formado de las siguientes secciones:

Código de inicio
Declaración de constantes simbólicas
Variables del programa
Código del programa
Código de terminación

Código De Inicio

El código de inicio de un programa en ensamblador para el 8086 tiene como propósito declarar los nombres de los segmentos en los que se almacenará el código del programa, los datos y la pila. También establece el modelo de memoria bajo el que se ensamblará el programa y el tamaño de la pila. El código de inicio consta de un conjunto de directivas. En el programa de ejemplo tenemos:

ideal

El modo **ideal** es una notación que emplea el ensamblador Turbo Assembler de Borland International para simplificar la sintaxis del código inicial de un programa. La directiva **ideal** le dice a Turbo Assembler que utilice el modo ideal. Si se omite se utiliza la notación empleada por la mayoría de los ensambladores, por ejemplo el de Microsoft.

dosseg

Esta directiva que forma parte del modo ideal, le instruye al ensamblador que nombre y ordene los segmentos del programa de la misma forma en que lo hacen los compiladores de alto nivel. Esto es adecuado para la mayoría de los programas en ensamblador y sobre todo en las rutinas de ensamblador que se van a ligar a un programa en alto nivel.

model

Esta directiva que debe ir después de la directiva **dosseg,** selecciona uno de los 6 modos diferentes que tiene Turbo Assenbler de generar el código de un programa dependiendo del tamaño que tiene el código y los datos. Estos modos llamados **modelos de memoria**, difieren en el tipo de apuntadores que el compilador genera por omisión y por lo tanto en la rapidez de ejecución del código. Los seis modelos de memoria son: Pequeñito (tiny), pequeño (small), medio (medium), compacto (compact), grande (large) y extenso (huge).

Modelo pequeñito (tiny)

En el modelo pequeñito, el código, los datos y la pila deben de estar en un solo segmento, esto es, el

programa completo no debe exceder a 64 KB. Todos los registros de segmento son inicializados al mismo valor y permanecen constantes durante la ejecución del programa. Bajo este modelo el compilador genera apuntadores cercanos. El código ejecutable producido es el más pequeño y rápido.

Modelo pequeño (small)

En el modelo pequeño, el código tiene su propio segmento y los datos y la pila están en otro segmento, esto es, el código no debe exceder a 64 KB ni tampoco los datos y pila. Los registros de segmento permanecen constantes durante la ejecución del programa. El código producido bajo este modelo utiliza también apuntadores cercanos y es igual de rápido que en el modelo pequeñito pero el tamaño del programa puede ser del doble. Normalmente debemos usar este modelo de memoria a menos que nuestro código o datos exceda los 64 KB.

Modelo medio (medium)

En el modelo medio, el código del programa puede ocupar varios segmentos pero los datos y la pila ocupan un solo segmento, esto es, el código puede exceder a 64 KB pero los datos y pila no. Por lo tanto, bajo este modelo el código utiliza apuntadores lejanos y los datos apuntadores cercanos. Debido a los apuntadores lejanos la velocidad de ejecución del programa se ve degradada. Los registros de los segmentos de datos y la pila se inicializan al mismo valor y permanecen constantes durante la ejecución del programa. Este modelo se debe usar cuando tengamos un programa de código extenso y pocos datos.

Modelo compacto (compact)

En el modelo compacto, el código del programa está limitado a un segmento, los datos pueden ocupar varios segmentos aunque el total de las variables estáticas no pueden exceder los 64 KB. Por lo tanto, bajo este modelo el código utiliza apuntadores cercanos y los datos apuntadores lejanos. Debido a los apuntadores lejanos también hay degradación en la velocidad de ejecución del programa. El registro del segmento de código permanece constante durante la ejecución del programa. Este modelo se debe usar cuando tengamos un programa de código pequeño y muchos datos.

Modelo grande (large)

En el modelo grande, tanto el código del programa como los datos puede ocupar varios segmentos, aunque el total de las variables estáticas no pueden exceder los 64 KB. Los apuntadores en este modelo son lejanos tanto para el código como para los datos. Esto hace que la velocidad de ejecución de un programa compilado bajo este modelo sea más lenta que cualquiera de las versiones anteriores. Este modelo se debe usar cuando tengamos un programa de código extenso y muchos datos.

Modelo extenso (huge).

En el modelo extenso, es igual que el modelo grande, sólo que el total de las variables estáticas si pueden exceder los 64 KB. Sin embargo cada variable estática en forma individual no puede exceder a 64 KB. Los apuntadores en este modelo son lejanos tanto para el código como para los datos. La velocidad de ejecución de un programa compilado bajo este modelo es más lenta que cualquiera de los otros modelos.

stack

Esta directiva reserva espacio de memoria para la pila del programa, un área de memoria que almacena valores temporales empleados por las subrutinas y las direcciones de regreso de las subrutinas.

Declaración de constantes simbólicas

Las constantes simbólicas son identificadores que le asociamos a constantes numéricas o a constantes cadenas. El uso de constantes simbólicas en un programa hace que sea más fácil de leer y modificar. Hay dos formas de declarar constantes simbólicas: mediante la directiva **equ** y mediante la directiva =. Por ejemplo:

```
filas   equ     10
cols    equ     5
tam     =       filas * cols
msj     equ     "Hola"
tam     =       0
```

Las directivas **equ** y = tienen las siguientes reglas:

- Las constantes simbólicas no son variables. Ni el nombre ni su valor asociado se almacena en el segmento de datos del programa.

- La directiva **equ** permite darle nombre a constantes numéricas, expresiones y cadenas. La directiva = sólo permite darle nombre a constantes numéricas y expresiones.
- No se le puede cambiar el valor asociado a un símbolo declarado con la directiva **equ**. En cambio a un símbolo declarado con la directiva = se le puede cambiar su valor tantas veces como se desee.

- Las directivas **equ** y = pueden aparecer en cualquier parte del programa. Pero es preferible colocarlas al principio ya que son más visibles.

- Las expresiones declaradas con = se evalúan inmediatamente, mientras que las expresiones declaradas con **equ** se evalúan hasta que el símbolo se usa.

Variables del programa

Las variables de un programa se declaran en el segmento de datos, por lo que la sección de declaración de variables empieza con la directiva **dataseg**. Esta directiva le indica a Turbo Assembler que almacene las variables en el segmento de datos del programa. Las variables pueden estar inicializadas o no. Los valores de las variables inicializadas se almacenan en el código del programa y se cargan en las variables al ejecutar el programa.

Para declarar variables podemos utilizar las directivas **db**, **dw**, **dd** o **dq**. La directiva **db** define una variable de un byte, la directiva **dw** define una variable de tipo palabra (dos bytes), la directiva **dd** define una variable de tipo palabra doble (cuatro bytes) y la directiva **dq** define una variable de tipo palabra cuádruple (ocho bytes). La sintaxis de estas directivas son:

```
nomVar    db   exp
nomVar    dw   exp
nomVar    dd   exp
nomVar    dq   exp
```

donde *nomVar* es el nombre de la variable y *exp* es una expresión constante cuyo valor se utiliza para inicializar la variable. Si la variable no se inicializa *exp* se substituye por un signo de interrogación, **?**. Por ejemplo

```
bdato   db      10h                ; Variable de un byte
                                   ; inicializada a 10h.

cresp   db      'A'                ; Variable de un byte
                                   ; inicializada a 65d.

bdatx   db      ?                  ; Variable de un byte
                                   ; no inicializada.

wdato   dw      0123h              ; Variable de tipo
                                   ; palabra inicializada
                                   ; a 0123h.

wdatx   dw      ?                  ; Variable de tipo
                                   ; palabra no
                                   ; inicializada.

ddato   dd      01234567h          ; Variable de tipo
                                   ; palabra doble
                                   ; inicializada a
                                   ; 01234567h.

ddatx   dd      ?                  ; Variable de tipo
                                   ; palabra doble
                                   ; no inicializada

qdato   dq      0123456789ABCDEFh  ; Variable de tipo
                                   ; palabra cuadruple
                                   ; inicializada a
```

```
                                        ; 0123456789abcdefh.

qdatx   dq      ?                       ; Variable de tipo
                                        ; palabra cuádruple
                                        ; no inicializada
```

Código del programa

En esta sección van las instrucciones que el microprocesador ejecutará al correr el programa. Todas las instrucciones van en el segmento de código, por lo que la sección de código del programa empieza con la directiva **codeseg**. Esta directiva le dice a Turbo Assembler que almacene las instrucciones en el segmento de código del programa.

El código del programa en la mayoría de los casos empieza con las siguientes líneas:

```
        codeseg
inicio:
        mov     ax, @data       ; Inicializa el segmento
        mov     ds, ax          ; de datos
```

La primera línea contiene la directiva **codeseg** mencionada anteriormente.

La segunda línea establece que *inicio* es una etiqueta asociada a la dirección de la primera instrucción del programa, que en este ejemplo aparece en la siguiente línea. Esta etiqueta es usada en la directiva **end** explicada en la siguiente sección.

En la tercera y cuarta línea se carga en el registro DS la dirección de inicio del segmento de datos del programa. El símbolo predefinido **@data** tiene el valor de esa dirección.
La instrucción **mov** mueve datos entre registros o entre registros y localidades de memoria. En este ejemplo la primera instrucción **mov** almacena la dirección del inicio del segmento de datos en el registro **AX** y la segunda instrucción **mov** mueve esa dirección del registro **AX** al registro de segmento **DS**. No se puede cargar una dirección directamente a uno de los registros de segmento, por lo que la instrucción

```
        mov     ds, @data        ; No válido
```

no es válida. La dirección debe cargarse primero a un registro de propósito general y luego de ahí moverse al registro de segmento.

También la mayoría de los programas que se van a ejecutar bajo el sistema operativo MSDOS deben terminar con las siguientes líneas de código:

```
salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h
```

El propósito de este código es llamar a una de las rutinas del sistema operativo que termina la ejecución

del programa y nos regresa al sistema operativo. Esta rutina que se conoce como una rutina de servicio a interrupción se verá en el Capítulo 8: INTERRUPCIONES. La llamada a esa rutina se hace mediante la instrucción:

```
int 21
```

Previamente se cargó en el registro **AH** el valor 04Ch que especifica que se desea terminar el programa y regresar al sistema operativo y en el registro **AL** el valor de la variable *codsal*, que es el código de salida del programa. El programa le regresa al sistema operativo el valor del código de salida en la variable de ambiente **errorlevel**. Por convención, un código de salida de cero denota que el programa terminó con éxito y un código diferente de cero indica que hubo un error durante la ejecución del programa. El valor del código identifica al error ocurrido.

En los programas en C también se acostumbra regresarle un código de salida al sistema operativo. esto se hace mediante la proposición return en la función *main*():

```
int main(void)
{
   ...

   return 0;
}
```

Código de terminación

Todo programa en ensamblador debe contener una directiva **end**, la cual marca el final del código fuente. Turbo Assembler ignora cualquier línea después de la directiva **end**. La sintaxis de la directiva **end** es la siguiente:

```
end  [etiqueta]
```

etiqueta es el nombre simbólico de la dirección de la instrucción por la que empezará a ejecutarse el programa. Si se omite, el programa empezará su ejecución en la primera instrucción del código fuente.

En un programa de un sólo módulo, es decir con un archivo con código fuente, la directiva **end** debe siempre especificar el inicio del programa. En un programa que consiste de más de un módulo, sólo la directiva **end** en el módulo que contiene la primera instrucción a ejecutar debe llevar una etiqueta. En los otros módulos la directiva **end** debe aparecer por sí sola.

Bibliografía

1. Abel, Peter. Lenguaje Ensamblador y Programación para PC IBM y Compatibles. Tercera Edición. Prentice-Hall Hispanoamericana, S. A. México. 1996.

2. Borland Int. Turbo Assembler Reference Guide. Version 1. Borland International. Scotts Valley, CA. 1988.

3. Brey, Barry B. Los microprocesadores Intel: 8086/8088, 80186, 80286, 80386 y 80486. Arquitectura, programación e interfaces. Tercera Edición. Prentice-Hall Hispanoamericana, S. A. México. 1995.

4. Godfrey, J. Terry. Lenguaje Ensamblador para Microcomputadoras IBM para Principiantes y Avanzados. Prentice-Hall Hispanoamericana, S. A. México. 1991.

5. Hyde, Randall. The Art of Assembly Language Programming. Este libro se encuentra como una serie de documentod PDF en el siguiente servidor FTP:

   ftp.cs.ucr.edu/pub/pc/ibmpcdir

6. Swan, Tom. Mastering Turbo Assembler. Hayden Books. Indiana, U.S.A. 1989.